

Online Examination Subsystem

Project Structure Report for Creational Design Patterns Assignment

1. Project Overview

This project implements an Online Examination Subsystem using multiple creational design patterns. The implementation is organized into separate packages for each assignment part: Factory Method, Abstract Factory, Builder, and Question Sourcing Integration.

```
Project root: Creational_Design_Pattern_Task/Attempt_1/src
```

2. High-Level Package Structure

Package	Purpose	Main Responsibility
Part_A	Factory Method Pattern	Creates different exam formats such as Practice Quiz, Midterm Exam, and Final Exam.
Part_B	Abstract Factory Pattern	Creates families of question-related objects: question entity, renderer, and evaluator.
Part_C	Builder Pattern	Builds flexible exam configuration objects through chained builder methods.
Part_D	Question Sourcing Integration	Supplies questions from manual, question bank, or faker generation strategies.
Main.java	Central Driver	Combines all packages and prints the final dashboard output.

3. Part A: Exam Provisioning

Part A defines the exam creation subsystem. It separates the client code from the concrete exam classes. The client asks an exam factory for an exam, and the selected concrete factory decides which exam object to instantiate.

Important Classes

Class / Interface	Role
Exam	Common interface for all exam products. It defines <code>getType()</code> and <code>displayInfo()</code> .
ExamFactory	Abstract creator. It declares <code>createExam()</code> .
PracticeQuiz, MidtermExam, FinalExam	Concrete exam products.
PracticeQuizFactory, MidtermExamFactory, FinalExamFactory	Concrete factories that create specific exam objects.

4. Part B: Question Subsystem

Part B models each question type as a family of related objects. A question is not only text: it also has a renderer for display and an evaluator for grading. The package supports MCQ, True/False, Essay, and Programming question families.

Core Interfaces

Interface	Responsibility
Question	Represents question data such as type, text, points, and difficulty.
QuestionRenderer	Displays a question in a clean console format.
QuestionEvaluator	Evaluates a submitted answer and returns the score.
QuestionFactory	Abstract factory interface. It creates a question, renderer, and evaluator.

Concrete Families

Family	Question	Renderer	Evaluator
MCQ	MCQQuestion	MCQRenderer	MCQEvaluator

True/False	TrueFalseQuestion	TrueFalseRenderer	TrueFalseEvaluator
Essay	EssayQuestion	EssayRenderer	EssayEvaluator
Programming	ProgrammingQuestion	ProgrammingRenderer	ProgrammingEvaluator

`BaseQuestion` is a helper abstract class that stores common question fields: text, points, and difficulty. It reduces repeated code across concrete question classes.

5. Part C: Exam Configuration Builder

Part C handles exam configuration. Instead of using a long constructor with many parameters, the project uses a builder with chained methods. This makes exam setup readable and flexible.

Important Classes

Class	Responsibility
<code>ExamConfiguration</code>	Final immutable configuration object. It stores title, duration, passing score, negative marking, shuffle, auto-submit, and calculator settings.
<code>ExamBuilder</code>	Builds an <code>ExamConfiguration</code> step by step using chained methods such as <code>setTitle()</code> , <code>setDuration()</code> , and <code>enableAutoSubmit()</code> .

Builder Usage Example

The final driver uses chained calls to build the exam configuration:

```
new ExamBuilder().setTitle("OOP Midterm Exam").setDuration(60).setPassingScore(50).enableNegativeMarking().enableQuestionShuffle().disableCalculator().enableAutoSubmit().build();
```

6. Part D: Question Sourcing Integration

Part D decouples question creation from the source of question data. The question factories can receive a `QuestionSource` through dependency injection. This allows the same factory to create questions from different sources.

Important Classes

Class / Interface	Responsibility
QuestionSource	Common interface for all question sourcing strategies. It defines <code>getQuestion()</code> and <code>getSourceName()</code> .
BankQuestionSource	Fetches questions from <code>QuestionBank</code> .
FakerQuestionSource	Generates mock questions using <code>QuestionFaker</code> .
ManualQuestionSource	Wraps a manually provided question. This supports the manual mode requirement.
QuestionBank	Stores predefined questions and returns a pseudo-random question by type.
QuestionFaker	Creates simulated academic questions at runtime.

7. Central Execution Flow

The root `Main.java` is the final orchestration driver. It performs the assignment workflow in this order:

1. Selects an `ExamFactory`, currently `MidtermExamFactory`.
2. Creates the exam using Factory Method.
3. Creates a `QuestionBank` and several `BankQuestionSource` objects.
4. Injects the sources into the concrete question factories.
5. Builds the runtime exam configuration using `ExamBuilder`.
6. Creates the final question list.
7. Prints a dashboard containing the exam type, configuration, sourcing strategy, and question components.

8. How the Project Follows the Required Patterns

8.1 Factory Method Pattern: Part A

Factory Method is followed because `ExamFactory` declares the creation method `createExam()`, while each subclass decides which concrete exam to instantiate. For example, `MidtermExamFactory` creates a `MidtermExam`, and `FinalExamFactory`

creates a `FinalExam`. The client depends on `ExamFactory` and `Exam`, not on direct constructor calls for every exam type.

8.2 Abstract Factory Pattern: Part B

Abstract Factory is followed because `QuestionFactory` creates a family of related products: `Question`, `QuestionRenderer`, and `QuestionEvaluator`. Each concrete factory produces compatible objects from the same family. For example, `MCQFactory` creates an MCQ question, MCQ renderer, and MCQ evaluator, while `EssayFactory` creates the Essay versions of those same roles.

8.3 Builder Pattern: Part C

Builder is followed because `ExamBuilder` constructs a complex `ExamConfiguration` object step by step. The configuration has many optional and variable fields, such as duration, passing score, negative marking, question shuffle, auto-submit, and calculator settings. The builder avoids constructor bloat and produces a completed immutable configuration object through `build()`.

8.4 Inversion of Control via Question Sources: Part D

The sourcing requirement is followed because concrete question factories can accept a `QuestionSource` object through dependency injection. This means the factory does not need to know whether the data comes from a question bank, faker generator, or manual source. The source is selected outside the factory and passed in, which keeps question origin flexible and replaceable.

9. Open/Closed Principle Support

The design supports the Open/Closed Principle because new exam types, question families, and question sources can be added mostly by creating new classes instead of modifying existing core logic. For example, a new Certification Exam could be added using a new concrete exam and factory. A Fill-in-the-Blank question could be added using a new question, renderer, evaluator, and factory family. A new database-backed question source could be added by implementing `QuestionSource`.